



Exercises session 2

Non-deterministic turing machines, PTIME.

1 Definitions

Definition 1.1 (Non-deterministic Turing Machine) A non-deterministic turing machine is defined as $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where

- Q is a finite set of states
- Σ is the input alphabet, with $\vdash, B \notin \Sigma$
- Γ is the tape alphabet, with $\Sigma \cup \{\vdash, B\} \subseteq \Gamma$
- $q_0 \in Q$ is the initial state
- $q_{\text{accept}}, q_{\text{reject}} \in Q$ are the accepting and rejecting states
- δ is a relation:

$$\delta \subseteq Q \times \Gamma \times Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$$

Definition 1.2 Let $t : \mathbb{N} \rightarrow \mathbb{N}$ be a function. We define the (deterministic) **time complexity class** $DTIME(t(n))$ as:

$$DTIME(t(n)) = \{L \mid L \text{ is decided by an } O(t(n)) \text{ time deterministic Turing machine}\}.$$

Definition 1.3 $PTIME$ (or **P**) is the class of all languages decidable in deterministic polynomial time on a single-tape TM

$$PTIME = \bigcup_k DTIME(n^k).$$

2 Session exercises

1. Lets introduce (recall?) boolean formulas

- **Syntax of boolean formulas:** let's consider a set of variables **Vars**: $\{x_1, \dots, x_n\}$ and operators $\vee, \wedge, \neg, \rightarrow, \leftrightarrow$, then we define boolean formulas recursively as
 - every variable x_i is a boolean formula

- if φ and ϕ are boolean formulas, then $(\neg\varphi)$, $(\varphi \wedge \phi)$, $(\varphi \vee \phi)$, $(\varphi \rightarrow \phi)$ and $(\varphi \leftrightarrow \phi)$ are also boolean formulas
- **Semantics of boolean formulas:** a truth assignment σ maps each variable x_i to either **true** (1) or **false** (0), formally $\sigma : \mathbf{Vars} \rightarrow \{0, 1\}$. We can evaluate a boolean formula ϕ using a truth assignment σ : we will denote this as $\phi(\sigma)$
 - the evaluation of a variable x_i corresponds to $\sigma(x_i)$
 - if ϕ_1 and ϕ_2 are boolean formulas, then if :
 - * $\varphi = (\neg\phi)$, then $\varphi(\sigma) = 1 - \phi(\sigma)$
 - * $\varphi = (\phi_1 \wedge \phi_2)$ then $\varphi(\sigma) = \min\{\phi_1(\sigma), \phi_2(\sigma)\}$
 - * $\varphi = (\phi_1 \vee \phi_2)$ then $\varphi(\sigma) = \max\{\phi_1(\sigma), \phi_2(\sigma)\}$
 - * $\varphi = (\phi_1 \rightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = 0$ or $\phi_2(\sigma) = 1$
 - * $\varphi = (\phi_1 \leftrightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = \phi_2(\sigma)$
 - **Satisfiability:** we say a boolean formula φ is satisfiable, if there exists a truth assignment σ such that $\varphi(\sigma) = 1$
 - **Conjunctive normal form:** we say a boolean formula is in conjunctive normal form (or CNF) if its a conjunction of disjunctions of literals. A literal is a variable or a negation of a variable. An example:

$$\varphi = (x_1 \vee \neg x_2) \wedge (x_1 \vee x_3)$$

In this exercise we will introduce the CNF-SAT problem, that will play an important role in future lessons of this course.

- (a) Given a CNF boolean formula φ and an assignment σ for such formula, give a TM that *verifies* whether $\varphi(\sigma) = 1$. What is the running time of such algorithm?
- (b) Now lets consider the following decision problem

$$\text{CNF-SAT} = \{\varphi : \varphi \text{ is a satisfiable CNF boolean formula} \}$$

Can we use the idea of the question above to solve this? What would be the running time?

- (c) What if we try using non-deterministic turing machines?
2. Show that the following language

$$\text{TRIANGLE} = \{\langle G \rangle : G \text{ has a triangle} \}$$

is in **P**

3. Analyze the following algorithm

```
Select a node in  $G$  and mark it
while no new nodes are marked do
  for each unmarked node  $u$  in  $G$  do
    mark  $u$  if it's connected to a marked node
  end for
end while
If all nodes are marked, return TRUE, otherwise, return FALSE
```

and conclude that

$$\text{CONNECTED} = \{ \langle G \rangle : G \text{ is a connected undirected graph} \}$$

is in **P**

4. Let's G be an undirected graph. A path is considered simple if it doesn't contain repeated nodes.

- Prove that

$$\text{SPATH} = \{ \langle G, a, b, k \rangle : \text{there's a simple path from } a \text{ to } b \text{ of length at most } k \}$$

is in **P**

- Is the following language also in **P**?

$$\text{LPATH} = \{ \langle G, a, b, k \rangle : \text{there's a simple path from } a \text{ to } b \text{ of length at least } k \}$$

5. Let's introduce the SUBSET-SUM problem: given a set of integers $\mathcal{A} = \{a_1, \dots, a_n\}$ and another integer K represented in binary. Is there a subset of $\mathcal{A}$ that sums exactly K ?

- Is this language in **P**?
- Now let's think of the same problem, but the integers are represented in unary. How does our complexity analysis change?